

Java Backend Architecture

Interview Series

Chapter 18.9

Nginx vs Alternatives

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Java Backend Architecture Interview Series

Chapter 18.9 – Nginx vs Alternatives

Overview

Senior engineers must evaluate Nginx against HAProxy, Traefik, and Envoy based on use case: Nginx excels as a general-purpose web server and reverse proxy; HAProxy as a pure high-performance load balancer; Traefik for cloud-native auto-discovery; and Envoy as the backbone of service meshes. Understanding each tool's strengths, limitations, and Java ecosystem integrations informs correct architectural decisions.

Key Definitions

Term	Definition
Nginx	Event-driven web server + reverse proxy. Combines HTTP server, cache, and load balancer.
HAProxy	High-performance L4/L7 load balancer. Pure proxy (no web server). Excellent health checks.
Traefik	Cloud-native proxy with automatic route discovery via Docker labels / K8s annotations.
Envoy	L7 proxy with xDS dynamic config (no reload), gRPC-first, service mesh sidecar (Istio).
xDS API	Envoy's dynamic configuration API. LDS (listeners), RDS (routes), CDS (clusters), EDS (endpoints).
Service mesh	Sidecar proxies (Envoy) alongside each Java pod handling service-to-service traffic.
VRRP / keepalived	Virtual Router Redundancy Protocol: active-active HA for HAProxy without external LB.
OpenResty	Nginx + LuaJIT. Enables dynamic scripting (JWT validation, dynamic routing) in Nginx.
Istio	Service mesh control plane using Envoy sidecars. Provides mTLS, traffic management, observability.
Middleware (Traefik)	Traefik's request transformation pipeline: rate limiting, auth, compression, retry.

Diagram 1: Tool Selection Decision Tree

```

Need to front Java microservices?
|
+-- Need web server (serve static files) + reverse proxy?
|   -> Nginx (Swiss Army knife, familiar, well-documented)
|
+-- Need pure maximum-throughput L4/L7 load balancing?
|   -> HAProxy (best health checks, ACL routing, stats)
|
+-- Running on Kubernetes / Docker with auto-discovery?
|   -> Traefik (reads K8s annotations automatically)
|
+-- Building service mesh / need per-request observability?
|   -> Envoy + Istio (xDS dynamic config, L7 metrics per call)
|
+-- Need Nginx + dynamic Lua scripting (JWT auth, WAF)?
    -> OpenResty (Nginx core + LuaJIT modules)
  
```

Diagram 2: Envoy Service Mesh vs Nginx Reverse Proxy

Nginx (edge proxy): External Client v [Nginx] v Java Services (1 proxy at edge)	Envoy (sidecar mesh): Java Pod A +-----+ App Container :8080 +-----+ Envoy Sidecar :15001 (egress) +-----+ mTLS v Java Pod B +-----+ Envoy Sidecar :15006 (ingress) +-----+ App Container +-----+ (proxy at every pod)
---	--

Code Examples

HAProxy Config for Java Backend:

```
frontend java_api_front
  bind *:80
  bind *:443 ssl crt /etc/haproxy/certs/
  mode http
  default_backend java_api_back

  # ACL-based routing
  acl is_api path_beg /api/
  use_backend java_api_back if is_api

backend java_api_back
  mode http
  balance leastconn
  option httpchk GET /actuator/health
  http-check expect status 200
  default-server inter 5s fall 3 rise 2
  server app1 app1:8080 check
  server app2 app2:8080 check
```

Traefik Dynamic Discovery (Kubernetes):

```
# Traefik Ingress with annotations
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  annotations:
    traefik.ingress.kubernetes.io/router.entrypoints: websecure
    traefik.ingress.kubernetes.io/router.middlewares: auth@kubernetescrd
spec:
  rules:
    - host: api.example.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service: { name: java-api, port: { number: 8080 } }
  ---
# Traefik Middleware (rate limit)
apiVersion: traefik.io/v1alpha1
kind: Middleware
metadata: { name: rate-limit }
spec:
  rateLimit: { average: 100, burst: 200 }
```

Envoy xDS Dynamic Config Snippet:

```
# Envoy static config -> dynamic via xDS control plane
static_resources:
  listeners:
    - name: listener_0
      address: { socket_address: { address: 0.0.0.0, port_value: 10000 } }
      filter_chains:
        - filters:
            - name: envoy.filters.network.http_connection_manager
              typed_config:
                route_config:
                  virtual_hosts:
                    - name: java_backend
                      domains: ["*"]
                      routes:
                        - match: { prefix: "/api" }
                          route: { cluster: java_api_cluster }
```

Interview Q&A – Nginx vs Alternatives

Q1. When should you choose HAProxy over Nginx?

HAProxy excels at pure high-throughput load balancing: active health checks (HTTP/TCP/custom scripts), granular ACL-based routing, active-active HA via keepalived+VRRP without external LB, excellent stats page, connection rate limiting per frontend. Choose HAProxy when you need a pure L4/L7 LB without web serving capabilities, or when HAProxy's active health checking is superior to Nginx's passive-only health checks.

Q2. What makes Traefik better than Nginx for container environments?

Traefik auto-discovers routes from Docker labels and Kubernetes Ingress/CRD annotations -- no nginx.conf reload needed when services start/stop. Built-in Let's Encrypt certificate management. Dashboard for live traffic visualization. Middleware pipeline (rate limit, auth, compression, retry) via CRDs. Trade-off: less battle-tested at extreme throughput, fewer tuning knobs than Nginx for performance optimisation.

Q3. What is Envoy's xDS API and why is it transformative?

xDS is Envoy's dynamic configuration API: LDS (Listeners), RDS (Routes), CDS (Clusters), EDS (Endpoints). A control plane (Istio, Consul Connect) pushes config changes via gRPC streaming -- no reload, no dropped connections. Nginx requires reload for every upstream change. xDS enables Envoy to react to Kubernetes Endpoint changes in milliseconds, making it ideal for rapidly scaling Java microservices.

Q4. When is OpenResty the right choice?

OpenResty (Nginx + LuaJIT) when you need: JWT validation at edge without subrequest latency (lua-resty-jwt), dynamic routing decisions (Redis-backed feature flags), custom WAF rules (lua-resty-waf), API gateway features (request/response transformation). More powerful than plain Nginx but requires Lua expertise. Kong API Gateway is built on OpenResty.

Q5. How does Envoy service mesh compare to Nginx edge proxy?

Nginx edge: single proxy at cluster boundary -- simple, fast, mature. No visibility into service-to-service traffic. Envoy sidecar mesh: proxy injected alongside every Java pod -- provides mTLS between services, per-service-call metrics (latency, error rate), circuit breaking, retry logic, distributed tracing. Complexity cost: 2x pod count for sidecar containers, additional CPU/memory overhead per pod.

Q6. What are HAProxy's active health check capabilities vs Nginx?

HAProxy active health checks: HTTP probe to /actuator/health with customizable success criteria (status 200, response body match), TCP connect check, SSL handshake check. Checks run independently of real traffic -- server marked down before clients see errors. Nginx open-source: passive only (max_fails/fail_timeout on real requests). Nginx Plus: active health checks similar to HAProxy. For critical Java services, HAProxy's proactive health checking prevents any failed requests reaching clients.

Q7. How does Traefik middleware differ from Nginx modules?

Traefik middleware is a composable pipeline defined as Kubernetes CRDs or Docker labels: RateLimit, BasicAuth, Headers, Compress, Retry, CircuitBreaker, ForwardAuth. Chains are declarative and applied per-route. Nginx uses compiled modules (static or dynamic) -- adding new functionality requires module compilation. Traefik middleware is more dynamic and developer-friendly; Nginx modules are more performant and configurable.

Q8. What is the performance difference between Nginx, HAProxy, and Envoy?

HAProxy and Nginx have comparable throughput for pure proxying (100k+ RPS). HAProxy may edge out at high connection counts due to single-process architecture. Envoy has ~10-15% higher CPU overhead per request due to richer L7 processing and observability. For Java API serving, the difference is negligible compared to backend latency. Nginx has lower memory footprint for static file serving; HAProxy for pure proxying.

Q9. How does Kong API Gateway relate to Nginx?

Kong is built on OpenResty (Nginx + LuaJIT). It adds a plugin system (rate limiting, authentication, logging, transformation), declarative configuration via Kong's Admin API or deck, and a database (PostgreSQL/Cassandra) or DB-less mode. Kong provides API gateway features (API key management, OAuth2, developer portal) as plugins, abstracting away raw Nginx/Lua configuration. Good choice for Java API management platforms.

Q10. When would you use Nginx and Envoy together?

Common pattern: Nginx at the edge (TLS termination, static serving, WAF, rate limiting) + Envoy service mesh inside the cluster (service-to-service mTLS, circuit breaking, observability). Nginx handles North-South traffic (external clients to cluster); Envoy/Istio handles East-West traffic (Java service A to service B). Each tool used for what it does best, avoiding complexity overload on

either.